

PROMPTS PARA AGENTES DE CODIFICACIÓN

Del PoC al MVP, con Claude Code, Codex o Cursor

Diez prompts listos para copiar y adaptar, en el mismo orden en que los usarás durante la sesión — desde la auditoría del PoC hasta el despliegue opcional en Vercel.

Un prompt por hito, no un prompt para todo

Claude Code, Codex, Cursor y agentes similares pueden acelerar el trabajo de construir el MVP, pero no reemplazan las decisiones de diseño que ya tomaste en tus Fichas 1, 2 y 3. Esta guía reúne, en el orden en que realmente se usan, los diez prompts de la sección 9 de la Guía de transición — cada uno pensado para un hito concreto del proceso, no para "hacerlo todo de una vez".

ANTES DE EMPEZAR

Ten a mano tu Ficha 1 (caso de uso), Ficha 2 (arquitectura) y Ficha 3 (datos y ETL), además del notebook de tu PoC. Varios prompts te piden pegarlas directamente.

CÓMO COPIAR CADA PROMPT

Cada prompt trae marcadores entre corchetes — [ASÍ] — que debes reemplazar con las decisiones reales de tu equipo antes de enviarlo. Un prompt que podría servir para cualquier proyecto todavía no está listo.

Tres hábitos que no cambian entre prompts

- ☐ Pide primero un plan y espera tu aprobación — recién después, cambios pequeños.
- ☐ Revisa el diff, ejecuta la aplicación y valida los criterios de aceptación antes del siguiente hito.
- ☐ Nunca dejes que el agente declare que algo funciona sin haber ejecutado la prueba correspondiente.

QUÉ ENCONTRARÁS EN CADA PASO

Dónde encaja en el proceso, cuándo usarlo, y el prompt completo listo para copiar y adaptar — el mismo texto de la Guía de transición, sin resumir.

CLAUDE.md / AGENTS.md

Copia esto en un archivo de proyecto — **CLAUDE.md** para Claude Code, **AGENTS.md** para Codex y otros agentes compatibles — con las decisiones permanentes del equipo. Claude Code lee CLAUDE.md al iniciar una sesión; Codex y herramientas similares leen AGENTS.md.

CLAUDE.MD / AGENTS.MD

```
# Contexto de proyecto: adapta y guarda como CLAUDE.md o AGENTS.md

## Producto
Somos el equipo [NOMBRE]. Construimos un MVP para [USUARIO] que resuelve [PROBLEMA].

## Alcance
El MVP SÍ hace: [CAPACIDAD PRINCIPAL].
El MVP NO hace: [LÍMITES Y ACCIONES PROHIBIDAS].

## Arquitectura
- Frontend: HTML + JS estático en index.html / static/ (sin frameworks)
- Backend: FastAPI en api/chat.py, expone POST /api/chat
- Orquestador: agent.py (LangChain, create_agent, MultiServerMCPClient)
- Servidor de herramientas: FastMCP en api/mcp.py, stateless_http=True
- Fuente de datos: [CSV/SQLite/API/RAG]
- Despliegue opcional: Vercel – no lo intenten hasta que todo funcione en local

## Reglas de desarrollo
- No inventes paquetes, APIs ni parámetros.
- Lee primero los archivos relevantes y propone un plan antes de editar.
- No borres archivos, datos o configuraciones sin pedir confirmación.
- No agregues dependencias sin justificar por qué son necesarias.
- Mantén una sola responsabilidad por archivo.
- Agrega o actualiza pruebas cuando cambies comportamiento.
- Nunca escribas secretos en código, logs, README o commits.
- Ejecuta las pruebas y reporta exactamente qué funcionó y qué falta.

## Criterios de aceptación
1. [CASO FELIZ]
2. [CASO LÍMITE]
3. [FUERA DE ALCANCE]
```

MODO DE TRABAJO RECOMENDADO

Pide primero: "analiza, explica el plan y espera mi aprobación". Luego solicita cambios pequeños. Revisa el diff, ejecuta la aplicación y valida los criterios de aceptación antes de pasar al siguiente hito.

Auditoría del PoC antes de migrar

Úsalo primero, antes de escribir una sola línea de código. Le pasas al agente tus tres fichas y tu PoC, y le pides un plan — no código todavía.

PROMPT 1 · COPIA Y ADAPTA

Actúa como arquitecto de software pedagógico. Analiza mi PoC y las decisiones adjuntas.

CASO DE USO:

[PEGA LA FICHA 1]

ARQUITECTURA:

[PEGA LA FICHA 2]

DATOS Y ETL:

[PEGA LA FICHA 3]

POC ACTUAL:

[PEGA O DESCRIBE EL NOTEBOOK Y LAS TOOLS]

Mi ruta del PoC fue: [Ruta A / B / C – ver sección 3 de la guía].

Objetivo: proponer el MVP mínimo con FastAPI + FastMCP + frontend estático.

No escribas código todavía. Entrega:

1. La capacidad única que debo migrar primero y por qué.
2. El contrato de la tool: nombre, descripción, parámetros, validaciones, estructura de respuesta y límites.
3. La fuente de datos que usará el MVP y riesgos de calidad o privacidad.
4. Tres criterios de aceptación: feliz, límite y fuera de alcance.
5. La estructura de archivos mínima (ver sección 5 de la guía).
6. Preguntas de decisión que aún debo resolver.

Crear el esqueleto sin sobrearquitectura

El primer prompt de construcción real. Pide un plan corto antes de tocar archivos, y limita exactamente qué estructura puede crear — nada más.

PROMPT 2 · COPIA Y ADAPTA

Trabaja dentro del repositorio actual. Antes de modificar archivos, inspecciona la estructura y responde con un plan de máximo 8 pasos. Espera mi aprobación.

Luego crea solo esta estructura:

- mcp_server.py
- agent.py
- config.py
- prompts.py
- index.html
- static/app.js
- static/style.css
- api/__init__.py
- api/mcp.py
- api/chat.py
- requirements.txt
- .env.example
- README.md
- data/
- tests/test_smoke.py

Restricciones:

- Es un MVP local; el despliegue en Vercel es un paso aparte y opcional.
- FastAPI para el backend, FastMCP en modo stateless_http=True para las tools.
- Frontend estático (HTML + JS), sin frameworks ni build step.
- No agregues Docker, bases remotas, autenticación compleja, colas ni microservicios.
- No agregues dependencias no justificadas.
- Incluye marcadores TODO donde falte información del dominio.
- Al terminar, muestra el árbol de archivos, explica cada uno y señala los comandos para ejecutar el proyecto.

Implementar una tool FastMCP segura en modo stateless

Para escribir la tool que formaliza la capacidad que ya validaste en el PoC. Define el contrato en la conversación antes de pedir código.

PROMPT 3 · COPIA Y ADAPTA

Implementa en `mcp_server.py` una tool FastMCP para esta capacidad:
[DESCRIBE LA CAPACIDAD]

Contrato deseado:

- Nombre: [NOMBRE_TOOL]
- Fuente: [CSV/SQLITE/API/DOCUMENTOS]
- Parámetros: [PARAMETROS]
- Resultado: [ESTRUCTURA ESPERADA]
- Límites: [NO MODIFICAR DATOS / NO DATOS PERSONALES / OTROS]

Requisitos:

1. Antes de escribir código, explica el contrato en una tabla.
2. Usa tipos Python claros y docstring orientado a decidir cuándo usar la tool.
3. Valida entradas vacías, formatos inválidos y ausencia de datos.
4. Devuelve resultados estructurados con ok, fuente y advertencias cuando corresponda.
5. No expongas secretos ni datos sensibles.
6. Expón la app con `mcp.http_app(path="/", stateless_http=True)` en `api/mcp.py`.
7. Incluye un ejemplo de datos de demostración y una prueba manual.
8. Ejecuta el servidor y reporta la URL esperada.

No agregues operaciones de escritura ni permisos amplios.

Integrar el agente con MCP

Cuando la tool del paso anterior ya corre. El agente debe descubrirla por protocolo, nunca importar la función directamente.

PROMPT 4 · COPIA Y ADAPTA

Revisa `mcp_server.py`, `api/mcp.py`, `config.py` y `prompts.py`. Luego implementa o corrige `agent.py` para que:

- Reutilice el bloque de ChatOpenAI + OpenRouter que ya está probado en el PoC (o el del proveedor que el equipo haya elegido).
- Cree un cliente `MultiServerMCPClient` conectado por HTTP a `MCP_URL`.
- Descubra las tools con `client.get_tools()`.
- Construya un agente con `create_agent`.
- Exponga una función async `responder(pregunta: str) -> str`.

Reglas:

- No uses patrones obsoletos como `AgentExecutor`, `initialize_agent` o `create_react_agent`.
- No asumas nombres de paquetes o parámetros: verifica la documentación o los imports existentes.
- Si falta el paquete del proveedor de modelo, indícalo con claridad en `requirements.txt`.
- Añade manejo de errores comprensible para el usuario y detalle técnico útil para desarrollo.
- Al final, ejecuta una consulta de humo y muestra la evidencia de que la tool MCP fue descubierta.

Crear el backend FastAPI y el frontend estático

Con el agente ya respondiendo, este prompt construye la única pieza que ve la persona usuaria: el chat en el navegador.

PROMPT 5 · COPIA Y ADAPTA

Implementa `api/chat.py` y la interfaz estática (`index.html`, `static/app.js`, `static/style.css`) para el agente existente.

El backend debe:

- Servir `index.html` en GET `/`.
- Montar `static/` como archivos estáticos.
- Exponer POST `/api/chat` que reciba `{"pregunta": str}` y llame a `responder()` de `agent.py`.
- Devolver errores como JSON claro, nunca como excepción sin manejar.

La interfaz debe:

- Mostrar título, breve explicación del alcance y advertencia de datos de demostración.
- Usar `fetch` a `/api/chat` (ruta relativa, sin hardcodear host ni puerto).
- Mantener un historial simple de mensajes en el DOM.
- Mostrar un estado de carga mientras se consulta el agente.
- Mostrar un error amigable si el backend o el servidor MCP no responden.
- No llamar directamente a ninguna fuente de datos.

No agregues autenticación, frameworks de frontend ni build step. Cuando termines, ejecuta `uvicorn api.chat:app --reload --port 8000` y describe las pruebas manuales que debo realizar.

Crear pruebas de aceptación

Cuando el flujo completo ya corre en local. Convierte tu política de incertidumbre en pruebas reproducibles, no en promesas.

PROMPT 6 · COPIA Y ADAPTA

Basado en este caso de uso y esta política de incertidumbre, define e implementa pruebas mínimas para el MVP.

CASO DE USO: [RESUMEN]

POLÍTICA DE INCERTIDUMBRE: [REGLAS]

TOOL DISPONIBLE: [NOMBRE Y CONTRATO]

Necesito:

1. Un caso feliz que requiera usar la tool correcta.
2. Un caso límite sin evidencia suficiente.
3. Un caso fuera de alcance.
4. Una prueba de entrada inválida hacia la tool.
5. Una prueba de indisponibilidad o error controlado de la fuente, si es posible sin complicar el MVP.

Primero entrega la matriz de pruebas. Después implementa solo las que sean reproducibles en este repositorio. Ejecuta las pruebas y no declares éxito si alguna falla.

Depurar un error sin reescribir todo

Para cuando algo se rompe. Este prompt frena al agente antes de que reescriba medio proyecto: primero diagnóstico, después tu aprobación.

PROMPT 7 · COPIA Y ADAPTA

Tengo este error al ejecutar [COMANDO]:
[PEGA EL ERROR COMPLETO]

Contexto del proyecto:

- Sistema: FastAPI + FastMCP + LangChain MCP Adapters.
- Archivo que falló: [ARCHIVO].
- Comando usado: [COMANDO].
- Cambio más reciente: [CAMBIO].

No modifiques código todavía. Haz lo siguiente:

1. Explica la causa probable en lenguaje claro.
2. Indica qué archivo(s) y líneas revisarías.
3. Propón el cambio mínimo y por qué.
4. Indica cómo verificar la corrección.
5. Espera mi confirmación antes de editar.

No reemplaces el proyecto ni introduzcas dependencias nuevas salvo que sea imprescindible y lo justifiques.

Revisión de seguridad y alcance

Una pasada de auditoría antes de mostrar el MVP a alguien más — o antes de intentar Vercel. Solo detecta y reporta; no corrige todavía.

PROMPT 8 · COPIA Y ADAPTA

Actúa como revisor de seguridad para un MVP educativo con MCP. Revisa este repositorio sin modificarlo todavía.

Busca específicamente:

- Secretos expuestos en archivos, logs, ejemplos o vercel.json.
- Tools que escriben, borran o modifican datos sin necesidad.
- Parámetros sin validación.
- Datos personales o sensibles en data/.
- Mensajes de error que filtren información innecesaria.
- Dependencias no justificadas.
- Riesgos de que el agente use una tool fuera de su alcance.

Entrega una tabla con: hallazgo, severidad, evidencia, cambio mínimo recomendado y si debe resolverse antes de la demo. Después espera mi aprobación para aplicar correcciones.

Preparar el proyecto para el despliegue opcional en Vercel

Solo cuando todo funciona en local. Prepara la versión unificada (api/index.py) sin tocar la lógica del agente, y te entrega los comandos — no despliega por ti.

PROMPT 9 · COPIA Y ADAPTA

Vamos a intentar el paso OPCIONAL de despliegue en Vercel descrito en la sección 8 de la guía. No toques agent.py, mcp_server.py, config.py, prompts.py ni el frontend salvo que sea estrictamente necesario.

Haz lo siguiente:

1. Crea api/index.py como versión unificada: monta mcp_server.mcp vía `mcp.http_app(path="/", stateless_http=True)` en /api/mcp dentro de la misma app FastAPI que sirve index.html, /static y /api/chat.
2. No borres api/chat.py ni api/mcp.py: muévelos a una carpeta local/ de respaldo.
3. Verifica que requirements.txt no tenga nada innecesario para el tamaño del bundle.
4. Genera o actualiza .env.example para reflejar que MCP_URL en producción apunta a `https://<dominio>.vercel.app/api/mcp`.
5. No ejecutes vercel deploy por mí: solo prepara el proyecto y dime los comandos exactos que debo correr, en orden.

Recuérdame verificar /api/mcp directamente en el navegador antes de probar el chat completo.

Revisión final antes de la entrega

El último prompt del proceso. Le pides al agente que actúe como evaluador externo, no como colaborador — sin tocar nada.

PROMPT 10 · COPIA Y ADAPTA

Revisa el repositorio como si fueras evaluador de un MVP académico. No edites archivos.

Evalúa:

1. ¿El proyecto se instala y ejecuta con las instrucciones del README?
2. ¿Existe un servidor FastMCP y una tool real con contrato claro?
3. ¿El agente descubre tools mediante MCP en lugar de importar la función directamente?
4. ¿La interfaz estática demuestra el flujo completo?
5. ¿El prompt contiene alcance y política de no invención?
6. ¿Hay casos feliz, límite y fuera de alcance, probados?
7. ¿Hay secretos o datos sensibles expuestos, en código o en configuración de Vercel?
8. ¿La estructura sigue siendo proporcional a un MVP?

Entrega una rúbrica de 0 a 2 por criterio, una lista priorizada de bloqueadores y una recomendación concreta para la próxima iteración.

Cómo adaptar estos prompts a cada herramienta

Los diez prompts anteriores funcionan como texto plano en cualquier agente de codificación. Lo que cambia es dónde vive el contexto persistente y cómo pides el plan antes de ejecutar.

Herramienta	Uso recomendado en este proyecto
Claude Code	Crea CLAUDE.md con arquitectura, comandos y criterios de aceptación. Pide plan antes de editar y revisa cada diff. Usa el terminal integrado para ejecutar las pruebas, no para adivinar resultados.
Codex	Mantén AGENTS.md con convenciones y comandos. Formula tareas concretas, pide que ejecute pruebas y que explique los cambios. Revisa el estado de Git antes de aceptar.
Cursor	Guarda las instrucciones del proyecto como Rules o AGENTS.md. Usa Plan Mode para cambios de arquitectura y Agent para implementar un paso ya aprobado.
Antigravity	Usa un Project o Workspace confiable, define Rules o Workflows persistentes y revisa los artifacts o cambios antes de aceptar ejecuciones. Reserva modos rápidos para tareas pequeñas y localizadas.
Otros agentes	Aplica el mismo patrón: contexto persistente (CLAUDE.md / AGENTS.md / Rules), plan breve antes de ejecutar, cambios pequeños, pruebas ejecutadas — no asumidas — y diff revisado antes de acciones sensibles.

En los cinco casos aplica el mismo patrón: contexto persistente (CLAUDE.md / AGENTS.md / Rules), plan breve antes de ejecutar, cambios pequeños, pruebas ejecutadas — no asumidas — y diff revisado antes de aprobar.

Reglas no negociables

Los agentes de codificación aceleran el trabajo, pero pueden editar archivos y ejecutar comandos. La responsabilidad de conservar el control sobre decisiones, permisos y datos sigue siendo tuya — en los diez prompts anteriores y en cualquier otro que escribas.

- ☐ No pegues claves API, tokens, credenciales, datos personales reales ni documentos sensibles en chats o prompts.
- ☐ Nunca comitas `.env`. Versiona solamente `.env.example`.
- ☐ Mantén los datos de demostración anonimizados y mínimos.
- ☐ Para el MVP, usa tools de solo lectura cuando sea posible.
- ☐ No des permisos de borrado, escritura masiva o administración a una tool solo por conveniencia.
- ☐ Revisa los diffs antes de aceptar cambios del agente de programación.
- ☐ Ejecuta comandos en el repositorio correcto y confirma rutas antes de borrar o limpiar archivos.
- ☐ Mantén Git activo y haz commits pequeños después de cada hito estable.
- ☐ No declares que algo funciona si no ejecutaste la prueba correspondiente.
- ☐ Si intentan el despliegue opcional, no dejen variables de entorno ni claves visibles en logs de Vercel.

RECUERDA

MCP no hace segura una capacidad por sí mismo, y un agente de codificación no reemplaza la revisión humana. La seguridad se construye con límites de las tools, validación de entradas, permisos mínimos, manejo de errores, control de secretos — y diffs que efectivamente lees antes de aceptar.

Esta guía complementa la "Guía de transición: Del PoC al MVP" (secciones 6 a 11) y la presentación "Del PoC al MVP" — úsalas en conjunto: la guía y el deck explican el qué y el por qué; estos prompts son el cómo, listo para copiar.